

Project 4: Resource Planning Ledger

CSCI-P532 Object Oriented Software Development Spring 2026

Overview

This project asks you to build a **Resource Planning Ledger** — a system for planning, tracking, and auditing the allocation of resources (people, equipment, consumables) across a portfolio of work plans, while maintaining a double-entry ledger of resource consumption. The domain is drawn from two chapters of *Analysis Patterns*:

- **Chapter 8 (Planning)** supplies: proposed and implemented actions, plans with action references and dependency graphs, protocols as reusable knowledge-level templates, resource allocations (general and specific), suspensions, and outcome/start functions.
- **Chapter 6 (Inventory and Accounting)** supplies: accounts, transactions, double-entry entries, posting rules, and the audit trail that records where every unit of resource came from and went.
- **Chapter 3 (Structure)** and **Chapter 4 (Composition)** supply the architectural discipline: the four-layer structure and composable design.

The learning goal is not feature completeness. It is to produce a design that can absorb the Week 2 requirement changes with minimal collateral damage.

Technology Stack

- **Backend:** Java 17, Spring Boot 3.x, Spring Data JPA
- **Database:** PostgreSQL (Docker Compose locally; Render.com managed PostgreSQL in production)
- **Frontend:** Plain HTML + CSS + JavaScript (*or optionally React / Angular*)
- **Build:** Maven
- **Containerisation:** Docker (multi-stage build)
- **Deployment:** Render.com free-tier Web Service + managed PostgreSQL
- **CI/CD:** GitHub Actions
- **Testing:** JUnit 5 + Mockito (`@ExtendWith(MockitoExtension.class)`)

Architectural Constraints (both weeks)

Your implementation **must** follow the four-layer architecture:

Layer	Spring stereotype	Responsibility
Client	<code>@RestController</code>	HTTP routing only; zero business logic
Manager	<code>@Service (Manager)</code>	Orchestrates use-case sequences
Engine	<code>@Service (Engine)</code>	Encapsulates one replaceable algorithm
ResourceAccess	<code>@Repository</code>	Atomic business verbs; no SQL in callers

REST controllers must **not** contain business logic. Engines must **not** call each other. ResourceAccess must **not** publish events.

WEEK 1 — Released now. Due end of Week 1.

Week 1: Core Planning and Ledger System

Domain Background

An **operations team** manages a portfolio of work **plans**. Each plan is a tree of **proposed actions** linked by dependency relationships. Actions consume **resources** (people and materials) and move through a well-defined lifecycle. Every resource movement is recorded in a double-entry **ledger**.

Week 1 Features

- F1. Protocol catalogue (knowledge level).** CRUD for protocols. Each protocol has: id, name, description, and an ordered list of *protocol steps*. Each step names a sub-protocol and may declare a dependency on one or more other steps within the same parent protocol. Protocols are the reusable templates from which plans are instantiated; they never change as a side-effect of plan execution.
- F2. Resource-type catalogue (knowledge level).** CRUD for resource types. Each resource type has: id, name, kind (**ASSET** or **CONSUMABLE**), unit of measure, and a linked *pool account*.
- F3. Plan management.** Create a plan from a protocol (the system generates one **ProposedAction** per step and copies the dependency structure) or from scratch. Display a plan's dependency tree in the UI. A plan is itself a **composite** of nodes: a node is either a leaf **ProposedAction** or a sub-plan (a **Plan** that groups a set of proposed actions). Both leaf and composite nodes share the same **PlanNode** interface, which exposes `getStatus()`, `getTotalAllocatedQuantity(ResourceType)`, and `accept(PlanNodeVisitor)`.
- F4. Action lifecycle.** A **ProposedAction** moves through states managed by a finite state machine. Legal transitions are:

From state	Event	To state
PROPOSED	implement()	IN_PROGRESS
PROPOSED	suspend()	SUSPENDED
PROPOSED	abandon()	ABANDONED
SUSPENDED	resume()	PROPOSED
SUSPENDED	abandon()	ABANDONED
IN_PROGRESS	complete()	COMPLETED
IN_PROGRESS	suspend()	SUSPENDED
IN_PROGRESS	abandon()	ABANDONED

Any attempt to trigger an illegal transition throws an **IllegalStateTransitionException**.

- F5. Proposed vs. implemented action.** When `implement()` fires, a separate **ImplementedAction** object is created linked to the original **ProposedAction**. The implemented action records actual start time, actual party, and actual location. Differences between the two objects are displayed in the UI.
- F6. Resource allocation.** Attach resource allocations (resource type, quantity, kind **GENERAL/SPECIFIC**) to actions. When specific, record the named asset identifier and an optional time period.
- F7. Ledger entries on completion.** When `complete()` fires, the system generates a double-entry transaction: one **Entry** withdraws the consumed quantity from the pool account; one deposits it into a per-action usage account. Both entries carry a *charged timestamp* (time of completion) and a *booked timestamp* (time of system recording).

- F8. Posting rule: over-consumption alert.** A posting rule watches each pool account. When an entry reduces the pool balance below zero, it appends an alert entry to a dedicated alert memo account. The rule fires eagerly.
- F9. Ledger view.** Display all entries for any account with booked date, charged date, amount, and originating action. Display current balances of all pool accounts.
- F10. Plan summary report.** A report generator traverses the composite plan tree using an **Iterator** that performs a depth-first traversal of **PlanNode** objects. The report prints each node's name, status, and total allocated quantity per resource type. Both leaf nodes and sub-plans are visited uniformly through the **PlanNode** interface.

Required Patterns — Week 1

State — ActionStateMachine. Each **ProposedAction** holds a reference to an **ActionState** object. The concrete state classes are **ProposedState**, **SuspendedState**, **InProgressState**, **CompletedState**, and **AbandonedState**. Each state class implements the **ActionState** interface:

```
interface ActionState {
    void implement(ActionContext ctx);
    void suspend(ActionContext ctx, String reason);
    void resume(ActionContext ctx);
    void complete(ActionContext ctx);
    void abandon(ActionContext ctx);
    String name();
}
```

A state that does not support a particular transition throws **IllegalStateTransitionException**. The **ActionContext** holds the **ProposedAction** entity and the reference to the **ActionManager** so that state transitions can trigger ledger operations and event publication. State objects are stateless singletons (one Spring bean per concrete state class).

Composite — PlanNode tree. Both **ProposedAction** (leaf) and **Plan** (composite) implement the **PlanNode** interface. **Plan** holds an ordered list of child **PlanNode** objects, which may themselves be **Plan** instances (sub-plans) or **ProposedAction** leaves. The **getStatus()** method on a composite **Plan** is derived: it returns **COMPLETED** only when all children are completed, **IN_PROGRESS** if any child is in progress or completed (but not all), **SUSPENDED** if any child is suspended and none is in progress, and **ABANDONED** if all children are abandoned. **getTotalAllocatedQuantity(ResourceType)** sums recursively over all leaf descendants.

Iterator — DepthFirstPlanIterator. **DepthFirstPlanIterator** implements **java.util.Iterator<PlanNode>** and traverses the composite **PlanNode** tree in depth-first order. It is used by the plan summary report (F10) and by the dependency-check logic to find all leaves in the correct topological order. Clients that iterate the tree must use this iterator; they must not recurse manually into children.

Template Method — AbstractLedgerEntryGenerator. The process of generating ledger entries on action completion follows a fixed skeleton defined in an abstract class:

```
abstract class AbstractLedgerEntryGenerator {

    // Template method -- final, defines the skeleton
    public final Transaction generateEntries(
        ImplementedAction action) {
        List<ResourceAllocation> allocs = selectAllocations(action);
        validate(allocs);
    }
}
```

```

    Transaction tx = createTransaction(action);
    for (ResourceAllocation a : allocs) {
        Entry withdrawal = buildWithdrawal(tx, a);
        Entry deposit    = buildDeposit(tx, a);
        postEntries(tx, withdrawal, deposit);
    }
    afterPost(tx);
    return tx;
}

protected abstract List<ResourceAllocation>
    selectAllocations(ImplementedAction action);
protected abstract void validate(
    List<ResourceAllocation> allocs);
protected Entry buildWithdrawal(
    Transaction tx, ResourceAllocation a) { /* default */ }
protected Entry buildDeposit(
    Transaction tx, ResourceAllocation a) { /* default */ }
protected void afterPost(Transaction tx) { /* hook, empty */ }

private Transaction createTransaction(ImplementedAction a)
    { /* final */ }
private void postEntries(Transaction tx,
    Entry w, Entry d) { /* final */ }
}

```

Week 1 provides one concrete subclass: `ConsumableLedgerEntryGenerator`, which selects only consumable resource allocations and validates that each has a positive quantity. The template method guarantees that conservation (balanced entries) is always checked in `postEntries`, which is `final` and cannot be overridden.

Domain Model — Week 1

Entity	Key attributes
<code>Protocol</code>	id, name, description
<code>ProtocolStep</code>	id, protocol, subProtocol, dependsOn (step names)
<code>ResourceType</code>	id, name, kind, unit, poolAccount
<code>Plan</code>	id, name, sourceProtocol (opt.), targetStartDate, children: List<PlanNode>
<code>ProposedAction</code>	id, name, protocol (opt.), party, timeRef, location, state: ActionState
<code>Suspension</code>	id, proposedAction, reason, startDate, endDate (opt.)
<code>ImplementedAction</code>	id, proposedAction, actualStart, actualParty, actualLocation, status
<code>ResourceAllocation</code>	id, action (poly.), resourceType, quantity, kind, assetId (opt.), timePeriod (opt.)
<code>Account</code>	id, name, kind (POOL/USAGE/ALERT_MEMO), resourceType
<code>Transaction</code>	id, description, createdAt
<code>Entry</code>	id, transaction, account, amount, chargedAt, bookedAt
<code>PostingRule</code>	id, triggerAccount, outputAccount, strategyType
<code>AuditLogEntry</code>	id, event, accountId, entryId, actionId, timestamp

API Sketch — Week 1

Method	Path	Description
GET/POST	/api/protocols	List / create protocols
GET/POST	/api/resource-types	List / create resource types
POST	/api/plans	Create plan
GET	/api/plans/{id}	Plan tree with statuses
GET	/api/plans/{id}/report	Depth-first summary report
POST	/api/actions/{id}/implement	Trigger implement()
POST	/api/actions/{id}/complete	Trigger complete()
POST	/api/actions/{id}/suspend	Trigger suspend()
POST	/api/actions/{id}/resume	Trigger resume()
POST	/api/actions/{id}/abandon	Trigger abandon()
POST	/api/actions/{id}/allocations	Attach resource allocation
GET	/api/accounts/{id}/entries	Ledger entries
GET	/api/accounts	All accounts with balances
GET	/api/audit-log	Audit log

UI Requirements — Week 1

- **Dashboard:** pool account balances; alert indicator for accounts below zero.
- **Plan view:** collapsible composite tree showing each node's name, type (leaf/plan), and derived status.
- **Action detail:** current state, legal transition buttons (only those permitted by the current state are enabled), resource allocations, and plan-vs-reality diff for implemented actions.
- **Ledger view:** entries per account with both timestamps.
- **Report page:** depth-first summary report for any selected plan.

Week 1 Deliverables

1. Working Spring Boot application, all Week 1 features passing.
2. **Design document** (1–2 pages): UML class diagram showing the four layers, the four patterns, and the knowledge/operational split. Justify each pattern choice.
3. **Unit tests:** minimum 15 tests covering all five state transitions (including illegal ones), the composite `getStatus()` derivation, the depth-first iterator order, and `ConsumableLedgerEntryGenerator`. No `@SpringBootTest`.
4. **Docker image** building and running on port 8080.
5. **Live URL** on Render.com with managed PostgreSQL.
6. **CI pipeline** (GitHub Actions): test → build → deploy.
7. **README** with Docker/Compose instructions, live URL, CI badge.

Week 1 Grading

Item	Points
Functional features (F1–F10)	14
Four patterns correctly applied	12
Design document + UML	8
Unit tests (≥ 15 , no SpringBootTest)	8
Docker + Render.com + PostgreSQL	4
CI/CD pipeline	4
Week 1 total	50

WEEK 2 — Released at the start of Week 2. Due end of Week 2.*Read this only after your Week 1 submission.*

Week 2: Requirement Changes

Each change is designed to land in a specific layer or pattern. The **Connection to design** paragraph names the component that should absorb most of the impact. Your Week 2 design retrospective must list, per change, which classes were modified, added, and untouched.

Change 1: New Action States and Guards

Rationale: The operations team has introduced a formal review gate. Before an action can move to `IN_PROGRESS`, a manager must approve it. Additionally, completed actions may now be *reopened* for correction.

New requirements

- Add two new states to the state machine:
 - `PENDING_APPROVAL`: entered from `PROPOSED` via a new `submitForApproval()` event. From this state only two transitions are legal: `approve()` → `IN_PROGRESS`, and `reject()` → `PROPOSED`.
 - `REOPENED`: entered from `COMPLETED` via a new `reopen()` event. From `REOPENED` the action may `complete()` again or `abandon()`. Reopening creates reversal ledger entries (equal and opposite to the completion entries) so the pool balance is restored.
- The direct `PROPOSED` → `IN_PROGRESS` transition is now **removed**. The only path to `IN_PROGRESS` is through `PENDING_APPROVAL`.
- Each new state is a new concrete `ActionState` implementation. All existing state classes are untouched except that `ProposedState.implement()` now throws `IllegalStateTransitionException` (the method body changes by one line).

Connection to design: Two new `ActionState` classes are added. `ActionManager` gains two new thin endpoint delegations. The Template Method skeleton in `AbstractLedgerEntryGenerator` is reused unchanged for the reversal entries by supplying a new `ReversalLedgerEntryGenerator` subclass that negates amounts. All other classes are untouched.

Change 2: New Ledger Entry Generator for Asset Allocations

Rationale: Until now only consumable allocations produce ledger entries. The team now requires that specific asset allocations also produce time-based ledger entries (recording hours used), with a different validation rule and a different post-hook that appends a utilisation note to the audit log.

New requirements

- Add a second concrete subclass of `AbstractLedgerEntryGenerator`: `AssetLedgerEntryGenerator`.
 - `selectAllocations()` returns only `SPECIFIC` asset allocations.
 - `validate()` checks that each allocation has a non-null time period and that the duration in hours is positive.
 - `buildWithdrawal()` and `buildDeposit()` use the duration in hours as the entry amount rather than a raw quantity.

- `afterPost()` (the hook) appends a utilisation record to the audit log for each processed asset.
- `ActionManager.complete()` now calls *both* generators in sequence: first `ConsumableLedgerEntryGenerator`, then `AssetLedgerEntryGenerator`. Each produces its own `Transaction`. The template method skeleton — including the conservation check in `postEntries` — runs identically for both.
- The ledger view gains a filter toggle: “Show consumable entries” / “Show asset entries” / “Show all”.

Connection to design: One new subclass of `AbstractLedgerEntryGenerator` is added; the abstract class and all other subclasses are untouched. `ActionManager.complete()` gains one additional method call. The Template Method pattern absorbs the entire change.

Change 3: Filtered and Lazy Plan Iterators

Rationale: Large plan trees contain hundreds of nodes. Different consumers need different views: the scheduler wants only `PROPOSED` leaves; the auditor wants only `COMPLETED` nodes; the resource planner wants only nodes with unresolved `GENERAL` allocations. Iterating the full tree every time is wasteful.

New requirements

- Add a `FilteredPlanIterator` that wraps a `DepthFirstPlanIterator` and accepts a `Predicate<PlanNode>`. `next()` skips nodes that do not satisfy the predicate. The underlying depth-first traversal is unchanged.
- Add a `LazySubtreeIterator` that accepts a root `PlanNode` and a depth limit. It performs a depth-first traversal but stops descending into a sub-plan once the depth limit is reached, yielding the sub-plan node itself rather than its children. This supports a “collapsed view” in the UI.
- All three iterators (`DepthFirstPlanIterator`, `FilteredPlanIterator`, `LazySubtreeIterator`) must implement `java.util.Iterator<PlanNode>` so that any consumer written against the interface works with all three without modification.
- The plan summary report (F10) is updated to accept an optional status filter query parameter; when provided it uses `FilteredPlanIterator` internally.
- The plan detail UI gains a depth slider; when set below maximum it renders the tree using `LazySubtreeIterator`.

Connection to design: Two new iterator classes are added. `DepthFirstPlanIterator` and all composite node classes are untouched. The report endpoint and the plan detail endpoint each gain one optional parameter.

Change 4: Composite Plan Metrics and a Visitor Hook

Rationale: Management wants real-time progress metrics for any node in the plan tree: percentage of leaf actions completed, total resource cost (quantity × unit cost), and a risk score based on the number of suspended or abandoned descendants. These calculations must work uniformly on both leaf nodes and sub-plans without the caller knowing which is which.

New requirements

- Add a `PlanNodeVisitor` interface:

```
interface PlanNodeVisitor {
    void visitLeaf(ProposedAction leaf);
    void visitComposite(Plan plan);
}
```


Add `accept(PlanNodeVisitor v)` to the `PlanNode` interface. Leaf nodes call `v.visitLeaf(this)`; composite nodes call `v.visitComposite(this)` and then recursively call `accept(v)` on each child.

- Implement three concrete visitors:
 1. `CompletionRatioVisitor`: counts total leaves and completed leaves; exposes `getRatio()`.
 2. `ResourceCostVisitor`: sums `quantity × unitCost` across all leaf allocations (unit cost stored on `ResourceType`); exposes `getTotalCost()`.
 3. `RiskScoreVisitor`: increments a counter for each leaf in `SUSPENDED` or `ABANDONED` state; exposes `getScore()`.
- A new `GET /api/plans/{id}/metrics` endpoint returns the three values for any node in the plan tree (identified by `id`, which may be a sub-plan or a leaf).
- The plan detail UI shows a metrics panel alongside the tree.

Connection to design: The Visitor is an extension of the Composite pattern already in place. `PlanNode` gains one method; the two existing concrete node classes each gain a one-line `accept()` implementation. The three visitor classes are entirely new. No other class changes.

Week 2 Deliverables

1. All four changes implemented and working.
2. **Design retrospective** (1 page): for each change, list classes modified, added, and untouched. If a change forced a modification you would not have expected, explain why and how you would redesign to prevent it.
3. **Updated unit tests**: at least 12 new tests covering `PendingApprovalState`, `ReopenedState` (including reversal entry creation), `AssetLedgerEntryGenerator`, `FilteredPlanIterator`, `LazySubtreeIterator`, and all three visitors.
4. Passing CI pipeline on the updated codebase.
5. Updated live Render.com deployment.

Week 2 Grading

Item	Points
Change 1: two new states + approval/reopen flow	10
Change 2: <code>AssetLedgerEntryGenerator</code> subclass	10
Change 3: filtered and lazy iterators	10
Change 4: visitor metrics on composite tree	8
Design retrospective (change-impact analysis)	8
New unit tests (≥ 12)	4
Week 2 total	50

Infrastructure Specification (Both Weeks)

PostgreSQL Configuration

Docker Compose for local development (docker-compose.yml):

```
version: "3.9"
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: rpl
      POSTGRES_USER: rpl
      POSTGRES_PASSWORD: rpl
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/rpl
      SPRING_DATASOURCE_USERNAME: rpl
      SPRING_DATASOURCE_PASSWORD: rpl
    depends_on:
      - db
volumes:
  pgdata:
```

In application.properties:

```
spring.datasource.url=
  ${SPRING_DATASOURCE_URL:jdbc:postgresql://localhost:5432/rpl}
spring.datasource.username=${SPRING_DATASOURCE_USERNAME:rpl}
spring.datasource.password=${SPRING_DATASOURCE_PASSWORD:rpl}
spring.jpa.database-platform=
  org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=update
```

Dockerfile (multi-stage)

```
# Stage 1 --- build
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn -q package -DskipTests

# Stage 2 --- run
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
```

```
ENTRYPOINT ["java","-jar","app.jar"]
```

GitHub Actions — .github/workflows/ci.yml

```
name: CI
```

```
on:
```

```
  push:
```

```
    branches: [ "**" ]
```

```
  pull_request:
```

```
    branches: [ main ]
```

```
jobs:
```

```
  test:
```

```
    runs-on: ubuntu-latest
```

```
    services:
```

```
      postgres:
```

```
        image: postgres:16-alpine
```

```
        env:
```

```
          POSTGRES_DB: rpl
```

```
          POSTGRES_USER: rpl
```

```
          POSTGRES_PASSWORD: rpl
```

```
        ports: [ "5432:5432" ]
```

```
        options: >-
```

```
          --health-cmd pg_isready
```

```
          --health-interval 10s
```

```
          --health-timeout 5s
```

```
          --health-retries 5
```

```
    steps:
```

```
      - uses: actions/checkout@v4
```

```
      - uses: actions/setup-java@v4
```

```
        with:
```

```
          java-version: '17'
```

```
          distribution: 'temurin'
```

```
      - run: mvn -q test
```

```
        env:
```

```
          SPRING_DATASOURCE_URL: jdbc:postgresql://localhost:5432/rpl
```

```
          SPRING_DATASOURCE_USERNAME: rpl
```

```
          SPRING_DATASOURCE_PASSWORD: rpl
```

```
      - uses: actions/upload-artifact@v4
```

```
        if: always()
```

```
        with:
```

```
          name: surefire-report
```

```
          path: target/surefire-reports/
```

```
build:
```

```
  needs: test
```

```
  runs-on: ubuntu-latest
```

```
  steps:
```

```
    - uses: actions/checkout@v4
```

```
    - uses: actions/setup-java@v4
```

```
      with:
```

```
        java-version: '17'
```

```
    distribution: 'temurin'
- run: mvn -q package -DskipTests
- run: docker build -t rpl .
```

```
deploy:
  needs: build
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  steps:
    - name: Trigger Render deploy
      run: curl -X POST ${ secrets.RENDER_DEPLOY_HOOK }
```

Store the Render deploy-hook URL as a GitHub secret named `RENDER_DEPLOY_HOOK`.

Render.com Setup

1. Create a **Web Service**; set environment to **Docker**; port **8080**.
2. Create a **PostgreSQL** database (free tier).
3. Copy the Internal Database URL into environment variables as `SPRING_DATASOURCE_URL`; set username and password variables.
4. Copy the deploy-hook URL into the GitHub secret.

Unit Testing Rules

- JUnit 5 + Mockito; `@ExtendWith(MockitoExtension.class)`.
- No `@SpringBootTest` in unit tests.
- Arrange / Act / Assert with comments.
- Inject `Clock` as a constructor dependency for determinism.
- Name tests: `methodName_scenario_expectedOutcome`.

README Requirements

- GitHub Actions CI badge.
- Live Render.com URL.
- `docker compose up` and standalone `docker run` commands.
- One paragraph per pattern explaining where it lives and why it was chosen.

Design Hints

State pattern: keep state objects stateless. Each `ActionState` implementation should be a Spring singleton bean. All mutable data lives in `ActionContext` (which wraps the JPA entity). This means adding a new state in Week 2 requires only a new class and a one-line change in whichever existing state class needs a new outgoing transition — nothing else changes.

Composite: persist the tree with a self-join. Map `PlanNode` as a JPA `@MappedSuperclass` with a parent nullable foreign key back to `Plan`. Use `@Inheritance(strategy = JOINED)` so `Plan` and `ProposedAction` each have their own table. The depth-first iterator loads children lazily to avoid fetching the whole tree on every request.

Iterator: keep it pure Java, not JPA. `DepthFirstPlanIterator` should operate on already-loaded in-memory `PlanNode` objects, not issue database queries inside `next()`. The `PlanManager` is responsible for loading the subtree before handing it to the iterator. This makes the iterator fully testable without a database.

Template Method: the hook is your extension point. The `afterPost()` hook in Week 1 is empty. In Week 2, `AssetLedgerEntryGenerator` overrides it to write the utilisation record. Never put business logic in `postEntries()` (which is `final`); only put it in overridable hooks or abstract methods. This is the design that makes Change 2 a single new subclass with zero changes to the base class.

Visitor extends Composite cleanly. The Visitor added in Change 4 is not a new pattern — it is an extension of the Composite already in place. If `PlanNode` already exposes `accept(PlanNodeVisitor)`, then the three metric visitors in Week 2 require no changes to existing node classes. The only addition is the one-line `accept()` method on each existing node class, which is minimal and predictable.

Knowledge vs. operational level. `Protocol`, `ProtocolStep`, and `ResourceType` are knowledge-level entities: created rarely, never modified by plan execution. `Plan`, `ProposedAction`, `ImplementedAction`, and `Entry` are operational. Keep these two groups in separate service and repository classes so that changes to the operational layer never touch the knowledge layer.